



Sistemas Operativos | Trabalho Prático

Marina Maria Maia Silva | N° 31029
Nelson Sérgio Moita Cruz | N° 31103
Nuno Miguel Fernandes Oliveira | N° 31550

Regime Pós-laboral

Orientação
Fernando Alexandre Peixoto Gomes

Ano letivo 2024/2025

Licenciatura em Engenharia de Sistemas Informáticos
Escola Superior de Tecnologia
Instituto Politécnico do Cávado e do Ave

ÍNDICE

1.	Introdução.....	3
1.1.	Análise do Problema	3
2.	Fundamentos Teóricos.....	4
2.1.	Gestão de Processos	4
2.1.1.	Processo.....	4
2.1.2.	Multiprogramação	4
2.1.3.	Tarefas (<i>Threads</i>).....	4
2.1.4.	Escalonamento de Processos	4
2.1.5.	Caracterização dos Sistemas Computacionais	5
2.1.6.	Escalonamento de Processos: Objetivos, Algoritmos e Estados.....	5
2.1.7.	Comportamento dos Processos	5
2.1.8.	Fatia de Tempo (<i>Quantum</i>)	6
2.1.9.	Desafetação Forçada do Processador	6
2.1.10.	Algoritmo Round Robin	6
2.1.11.	Escalonamento com Prioridades	6
2.2.	Gestão de Ficheiros	6
2.2.1.	System Calls.....	6
2.2.2.	Descritores de Ficheiros	7
2.3.	Sistemas de Ficheiros	9
2.3.1.	Tipos de Ficheiros	9
2.3.2.	Gestão Lógica de Armazenamento	10
2.3.3.	Análise de Sistemas de Ficheiros.....	10
3.	Implementação	11
3.1.	Parte 1: Implementação dos Comandos de Manipulação de Ficheiros	11
3.1.1.	Mostra Ficheiro.....	11
3.1.2.	Copia Ficheiro	12
3.1.3.	Acrescenta Origem Destino.....	13
3.1.4.	Conta Ficheiro	14
3.1.5.	Apaga Ficheiro	15
3.1.6.	Informa Ficheiro.....	16
3.1.7.	Lista Diretoria	17
3.2.	Parte 2 - Implementação de um interpretador de linha de comandos	19
3.2.1.	Interpretador com Funções da <i>LibC</i>	19
3.2.2.	Interpretador com <i>System Calls</i>	19
3.3.	Parte 3A - Gestão de Sistemas de Ficheiros.....	20
3.3.1.	Adicionar um Disco Virtual e Criar uma Partição	20
3.3.2.	Criação de Volumes Lógicos (LVM)	23
3.3.3.	Criação de Sistemas de Ficheiros	24
3.3.4.	Montagem dos Sistemas de Ficheiros.....	25
3.4.	Parte 3B - Análise ao Sistemas de Ficheiros.....	26
3.4.1.	Análise Geral do Sistema de Ficheiros.....	26
3.4.2.	Listagem de Ficheiros e Pastas.....	27
3.4.3.	Recuperação do Conteúdo de um Ficheiro	27
4.	Conclusão.....	28
5.	Bibliografia.....	28
	Anexo A-Enunciado do Trabalho Prático	29

1. Introdução

O presente relatório descreve o trabalho prático desenvolvido no âmbito da unidade curricular de Sistemas Operativos. Este projeto teve como objetivo central a aplicação prática dos conceitos de gestão de processos e de ficheiros, explorando a interação entre o utilizador e o sistema operativo através da implementação de um conjunto de comandos e de um interpretador de linha de comandos.

Na primeira fase do trabalho, o grupo colaborou em conjunto na implementação dos comandos de manipulação de ficheiros: mostra, copia, acrescenta e conta. Adicionalmente, o Nelson desenvolveu o comando apaga, a Marina implementou o comando informa e o Nuno ficou responsável pelo comando lista. Todos estes comandos foram implementados utilizando *system calls* para uma interação direta com o sistema operativo, permitindo aprofundar a compreensão dos mecanismos de acesso e manipulação de ficheiros a um nível de sistema, sem recurso a funções de bibliotecas de alto nível.

A segunda fase focou-se no desenvolvimento de um interpretador de linha de comandos personalizado. Nesta etapa, todos os membros do grupo contribuíram para a implementação da interação com as *system calls*, enquanto o Nelson se dedicou ao desenvolvimento da interação com a biblioteca LibC. Este interpretador facultava a execução de comandos introduzidos pelo utilizador, evidenciando os processos de criação e gestão de processos pelo sistema operativo.

Adicionalmente, o trabalho abordou a gestão de sistemas de ficheiros. O Nuno foi o principal responsável pela investigação e implementação da criação e configuração de partições, volumes e sistemas de ficheiros em máquina virtual (Parte 3A). O Nelson contribuiu na análise de um sistema de ficheiros existente, com o intuito de identificar a sua estrutura e conteúdo (Parte 3B).

A elaboração do presente relatório ficou a cargo da Marina.

Em suma, todas as etapas deste trabalho prático contaram com a participação ativa e colaborativa dos membros do grupo, permitindo uma compreensão abrangente dos temas abordados na unidade curricular.

1.1. Análise do Problema

O presente trabalho prático propõe um conjunto de desafios que abrangem a implementação de funcionalidades essenciais de um sistema operativo, nomeadamente no que concerne à gestão de ficheiros e processos (ver Anexo A-Enunciado do Trabalho Prático).

A primeira parte do enunciado centra-se na criação de um conjunto de comandos para manipulação de ficheiros. Esta etapa requer a utilização de *system calls* para executar operações como mostrar o conteúdo de um ficheiro, copiar ficheiros, acrescentar conteúdo, contar linhas, apagar ficheiros e obter informações sobre os mesmos.

A segunda parte do trabalho envolve o desenvolvimento de um interpretador de linha de comandos. Este interpretador deverá ser capaz de ler comandos do utilizador e executá-los, simulando a operação de uma *shell*. Este desafio implica a utilização de primitivas de execução de processos e a gestão do ciclo de vida dos processos criados.

A terceira parte do enunciado aborda a gestão e análise de sistemas de ficheiros. Esta parte divide-se em duas componentes: a primeira envolve a criação e configuração de partições, volumes lógicos e sistemas de ficheiros num servidor virtual, enquanto a

segunda consiste na análise de um sistema de ficheiros existente, com o objetivo de identificar a sua estrutura e conteúdo.

Globalmente, o trabalho prático visa consolidar os conhecimentos teóricos sobre sistemas operativos, através da sua aplicação prática em cenários de implementação e análise.

2. Fundamentos Teóricos

O presente trabalho prático fundamenta-se em conceitos cruciais da área de Sistemas Operativos, especificamente na gestão de processos e na gestão de ficheiros.

2.1. Gestão de Processos

2.1.1. Processo

Um processo, representa um programa em execução que utiliza os recursos do computador para cumprir a sua função. Estes recursos incluem o tempo do processador, a memória principal (onde o programa e os dados ficam guardados enquanto está a correr), e o "estado" do processo, que é toda a informação que o sistema operativo precisa de saber sobre ele para o gerir corretamente. É importante que cada processo funcione no seu próprio espaço, para que não haja confusões ou interferências com outros programas que estejam também a correr no mesmo computador. Além disso, o acesso aos recursos é controlado pelas permissões do utilizador que "lançou" o processo, o que ajuda a manter o sistema seguro.

2.1.2. Multiprogramação

A multiprogramação é a técnica de ter vários programas a correr "ao mesmo tempo" num computador. Num computador com apenas um processador (e um "núcleo"), o processador só consegue executar uma instrução de cada vez. Por isso, a multiprogramação nestes casos é mais uma ilusão de simultaneidade, chamada "pseudoconcorrência". A ideia é que o processador vai alternando entre os vários programas, dando um pouco de tempo a cada um, o que nos dá a sensação de que estão todos a correr ao mesmo tempo.

2.1.3. Tarefas (*Threads*)

Dentro de um processo, podemos ter várias "linhas de execução" a correr em simultâneo, chamadas tarefas ou *threads*. Estas *threads* partilham a mesma "área de trabalho" (espaço de endereçamento) do processo a que pertencem, o que facilita a comunicação entre elas. As *threads* são úteis para dividir um programa em várias partes que podem ser executadas em paralelo.

2.1.4. Escalonamento de Processos

O escalonamento de processos é o mecanismo que o sistema operativo usa para decidir qual dos processos pendentes deve ser executado a seguir. Como temos vários processos a querer usar o processador, é preciso haver regras para gerir a sua utilização. A "comutação de processos" (*context switch*) é a operação de trocar o processo que está a correr por outro.

2.1.5. Caracterização dos Sistemas Computacionais

A forma como o escalonamento é feito depende do tipo de sistema em que estamos a trabalhar. Podemos ter sistemas de:

- **Processamento por lotes (*batch processing*)**: em que os programas são executados um a seguir ao outro, sem qualquer interação com o utilizador.
- **Tempo partilhado**: em que vários utilizadores estão a utilizar o computador ao mesmo tempo, e o sistema operativo tem de gerir os recursos para que todos tenham uma boa experiência.
- **Tempo real**: em que o sistema tem de responder a eventos externos dentro de prazos muito rígidos.

2.1.6. Escalonamento de Processos: Objetivos, Algoritmos e Estados

O escalonamento de processos é o mecanismo pelo qual o sistema operativo determina qual o processo a ser executado em cada momento. Este processo visa garantir a utilização eficiente dos recursos do sistema, assegurando critérios como:

- **Justiça**, assegurando que todos os processos têm oportunidades equitativas de execução;
- **Desempenho**, otimizando métricas como o débito e o tempo de resposta;
- **Cumprimento de prazos**, em sistemas críticos ou de tempo real.

Para atingir estes objetivos, são aplicados diferentes algoritmos de escalonamento, dos quais se destacam:

- **FIFO (First In First Out)**: Executa os processos pela ordem de chegada;
- **Shortest Job First**: Dá prioridade aos processos mais curtos;
- **Round Robin**: Atribui a cada processo uma fatia de tempo fixa (quantum);
- **Escalonamento com prioridades**: Dá precedência a processos com maior prioridade definida.

Cada processo, ao longo do seu ciclo de vida, transita entre diferentes estados:

- **Executável** (pronto a correr),
- **Em execução** (a usar o processador),
- **Bloqueado** (a aguardar um evento externo).

Estas transições são geridas automaticamente pelo sistema operativo em função da disponibilidade do processador, das prioridades dos processos e da ocorrência de eventos como pedidos de entrada/saída. A correta gestão destes estados é fundamental para o equilíbrio e estabilidade do sistema.

2.1.7. Comportamento dos Processos

Os processos podem ser classificados em dois tipos:

- **IO-bound**: processos que passam mais tempo à espera de operações de entrada/saída (exemplo: ler do disco, receber dados da rede).
- **CPU-bound**: processos que passam mais tempo a fazer cálculos.

2.1.8. Fatia de Tempo (*Quantum*)

Em sistemas interativos, o processador é partilhado em "fatias de tempo" para garantir que todos os processos progridem. Um *quantum* é o tempo máximo que um processo pode correr seguidamente.

2.1.9. Desafetação Forçada do Processador

Em sistemas interativos, é importante que o sistema operativo consiga interromper um processo que está a correr para dar tempo a outros processos. Isto chama-se "desafetação forçada" (*preemptive scheduling*).

2.1.10. Algoritmo Round Robin

O algoritmo *Round Robin* é um algoritmo de escalonamento que dá a cada processo uma fatia igual de tempo do processador. Os processos são organizados numa fila, e o processador vai dando tempo a cada um por sua vez. Este algoritmo é justo, mas pode aumentar o tempo de resposta se houver muitos processos.

2.1.11. Escalonamento com Prioridades

O escalonamento com prioridades permite dar prioridade a certos tipos de processos. Por exemplo, processos interativos (que reagem ao utilizador) podem ter maior prioridade do que processos que fazem cálculos complexos.

2.2. Gestão de Ficheiros

A gestão de ficheiros constitui outra componente essencial dos sistemas operativos. Um ficheiro é uma coleção organizada de dados, armazenada num dispositivo de armazenamento não volátil, como um disco rígido. O sistema operativo providencia um conjunto de operações para a manipulação de ficheiros, incluindo a sua criação, leitura, escrita, eliminação e consulta de *metadados* (informação sobre o ficheiro, como tamanho, datas de acesso, permissões, etc.).

A implementação dos comandos de manipulação de ficheiros neste trabalho requer a utilização de *system calls* específicas para estas operações. A gestão de ficheiros também envolve conceitos como organização hierárquica em diretórios, permissões de acesso e diferentes tipos de ficheiros (e.g., ficheiros regulares, diretórios, links).

2.2.1. System Calls

Open:

A system call `open` (<https://linux.die.net/man/3/open>) é utilizada para abrir um ficheiro, estabelecendo uma ligação entre o ficheiro e um descritor de ficheiro que o processo pode utilizar para realizar operações de entrada/saída. A função `open` aceita diversos argumentos que especificam o modo como o ficheiro deve ser aberto, sendo a sua assinatura:

```
int open (const char *pathname, int flags, mode_t mode);
```

As opções mais comuns para o argumento *flags* incluem:

- `O_RDONLY`: Abre o ficheiro em modo de apenas leitura.
- `O_WRONLY`: Abre o ficheiro em modo de apenas escrita.
- `O_RDWR`: Abre o ficheiro em modo de leitura e escrita.
- `O_CREAT`: Cria o ficheiro se ele não existir.
- `O_TRUNC`: Trunca o ficheiro para comprimento zero se ele existir.
- `O_APPEND`: Abre o ficheiro em modo de acrescentar (escreve no fim do ficheiro).

O argumento *mode* é utilizado para definir as permissões do ficheiro quando este é criado com a *flag* `O_CREAT`. As permissões são especificadas utilizando uma combinação de constantes:

- `S_IRUSR`: Permissão de leitura para o dono do ficheiro.
- `S_IWUSR`: Permissão de escrita para o dono do ficheiro.
- `S_IXUSR`: Permissão de execução para o dono do ficheiro.
- `S_IRGRP`: Permissão de leitura para o grupo do ficheiro.
- `S_IWGRP`: Permissão de escrita para o grupo do ficheiro.
- `S_IXGRP`: Permissão de execução para o grupo do ficheiro.
- `S_IROTH`: Permissão de leitura para outros utilizadores.
- `S_IWOTH`: Permissão de escrita para outros utilizadores.
- `S_IXOTH`: Permissão de execução para outros utilizadores.

2.2.2. Descritores de Ficheiros

Os descritores de ficheiro são inteiros não negativos que o sistema operativo utiliza para referenciar um ficheiro aberto ou um canal de entrada/saída. Em programação C, e ao nível das *system calls*, existem constantes que representam estes descritores padrão da consola, facilitando a sua utilização e tornando o código mais legível:

- `0 (STDIN_FILENO)`: Este descritor, representado pela constante `STDIN_FILENO`, refere-se ao canal de entrada padrão. Por omissão, a entrada padrão é o teclado, de onde o programa lê os dados.
- `1 (STDOUT_FILENO)`: Este descritor, representado pela constante `STDOUT_FILENO`, refere-se ao canal de saída padrão. Por omissão, a saída padrão é o ecrã, para onde o programa escreve os seus resultados.
- `2 (STDERR_FILENO)`: Este descritor, representado pela constante `STDERR_FILENO`, refere-se ao canal de saída de erros. Por omissão, a saída de erros também é o ecrã, sendo utilizado para apresentar mensagens de erro e diagnósticos.

Ao contrário das funções da *LibC* como *fopen*, *fread* e *fwrite*, as *system calls* como *read* e *write* operam diretamente com os descritores de ficheiro, utilizando as constantes `STDIN_FILENO`, `STDOUT_FILENO` e `STDERR_FILENO` para referenciar os canais padrão.

Para além da *open*, que estabelece a ligação ao ficheiro, existem várias outras *system calls* essenciais para manipular o seu conteúdo e a sua existência no sistema:

Read

A system call *read* permite ler dados de um ficheiro já aberto para um buffer na memória do processo. A sua assinatura é:

```
ssize_t read(int fd, void *buf, size_t count);
```

- *fd* é o descritor do ficheiro obtido com *open*.
- *buf* é o endereço onde os dados serão armazenados.
- *count* indica o número máximo de bytes a ler.

Esta função retorna o número real de bytes lidos ou -1 em caso de erro. A leitura é sequencial e começa na posição atual do apontador do ficheiro.

Write

A system call *write* escreve dados do buffer para o ficheiro, podendo sobrescrever ou acrescentar, dependendo da posição atual do apontador e das flags usadas na abertura do ficheiro. A assinatura é:

```
ssize_t write(int fd, const void *buf, size_t count);
```

Os parâmetros são semelhantes ao *read*, mas neste caso os dados são copiados do buffer para o ficheiro. Retorna o número de bytes efetivamente escritos ou -1 em caso de erro.

Close

Para terminar a manipulação de um ficheiro, usa-se *close*, que fecha o descritor, libertando recursos do sistema. É obrigatório fechar todos os descritores abertos para evitar fugas de memória e inconsistências. A assinatura é:

```
int close(int fd);
```

Retorna 0 em sucesso ou -1 se houver erro.

Unlink

Para eliminar um ficheiro do sistema, utiliza-se *unlink*, que remove a entrada do ficheiro na diretoria. Caso existam descritores abertos para esse ficheiro, o espaço só é libertado após o último descritor ser fechado.

```
int unlink(const char *pathname);
```

Retorna 0 em caso de sucesso ou -1 em erro.

Com estas *system calls*, é possível controlar a criação, leitura, escrita, navegação e eliminação de ficheiros, formando a base das operações de I/O em sistemas UNIX/Linux. Estas *system calls* combinam-se para permitir desde simples leituras e escritas sequenciais até manipulações avançadas de ficheiros, garantindo flexibilidade e controlo detalhado sobre o acesso aos dados.

2.3. Sistemas de Ficheiros

Um sistema de ficheiros define a estrutura lógica e a organização dos dados num dispositivo de armazenamento. Diferentes sistemas de ficheiros, como o ext3 e o ext4, implementam distintas estruturas de dados e algoritmos para gerir o espaço em disco, alocar ficheiros e diretórios, e otimizar o acesso aos dados. As escolhas de um sistema de ficheiros influenciam o desempenho, a fiabilidade e a capacidade de armazenamento.

A terceira parte deste trabalho prático explora a criação e manipulação de sistemas de ficheiros, bem como a análise da sua estrutura interna. Esta análise inclui a compreensão de conceitos como blocos de dados, *inodes* (estruturas de dados que armazenam *metadados* sobre os ficheiros), *superbloco* (que contém informação sobre o sistema de ficheiros como um todo) e a organização dos dados em disco.

2.3.1. Tipos de Ficheiros

Ficheiros Regulares: Estes ficheiros armazenam o conteúdo propriamente dito da informação do utilizador. Podem conter dados em diversos formatos, como texto, imagens, áudio, vídeo, código executável de programas, ou qualquer outro tipo de dados.

Diretórios: Estes ficheiros especiais são utilizados para estruturar o sistema de ficheiros de forma hierárquica. Um diretório atua como um contentor, podendo conter outros ficheiros (regulares ou especiais) e outros diretórios (subdiretórios). Esta organização em árvore permite agrupar e organizar os ficheiros de forma lógica, facilitando a sua localização e gestão. Os diretórios contêm metadados que descrevem os ficheiros e subdiretórios que lhes estão subordinados, como nomes, localizações e atributos.

Links: Os links são ficheiros especiais que estabelecem ligações entre ficheiros. Existem dois tipos principais:

- **Links Simbólicos (Soft Links):** Um link simbólico contém um caminho (string) para outro ficheiro ou diretório. Atua como um atalho. A sua resolução é feita pelo sistema operativo no momento do acesso. Se o ficheiro ou diretório alvo for movido ou eliminado, o link simbólico deixa de ser válido.
- **Links Físicos (Hard Links):** Um link físico é uma entrada adicional no sistema de ficheiros que referencia os mesmos dados (inode) que outro ficheiro. Ambos os nomes são, na prática, nomes equivalentes para o mesmo conteúdo. Um ficheiro só é efetivamente removido do sistema de ficheiros quando todas as suas ligações físicas são eliminadas.

Ficheiros Especiais: Estes ficheiros fornecem uma interface para dispositivos de hardware e outros recursos do sistema, permitindo que as aplicações interajam com eles através das mesmas operações de leitura e escrita utilizadas para ficheiros regulares. Dividem-se em:

- **Ficheiros Especiais orientados ao caractere:** Estes ficheiros representam dispositivos que transferem dados de forma sequencial, caractere a caractere. Exemplos comuns incluem terminais, teclados, ratos e portas série.
- **Ficheiros Especiais orientados ao bloco:** Estes ficheiros representam dispositivos que transferem dados em blocos. Discos rígidos, unidades SSD, unidades USB e unidades de CD-ROM são exemplos típicos.

Named Pipes (FIFOs): As Named Pipes são um tipo especial de ficheiro utilizado para comunicação entre processos. Permitem que processos não relacionados comuniquem entre si, lendo e escrevendo dados para o pipe como se fosse um ficheiro regular. Os dados escritos por um processo são lidos por outro, fornecendo um mecanismo de comunicação sincronizada.

2.3.2. Gestão Lógica de Armazenamento

A gestão lógica de armazenamento permite criar estruturas flexíveis e escaláveis para organizar o espaço em disco. Esta abordagem é especialmente relevante em ambientes onde é necessário redimensionar volumes ou separar logicamente dados com diferentes características.

Um dos mecanismos mais utilizados para este fim é o Logical Volume Manager (LVM), que abstrai a gestão do espaço em disco através de três componentes principais:

- **Volume Físico (PV – Physical Volume):** Representa uma partição ou disco completo que será utilizado pelo LVM. Um PV pode ser criado sobre um dispositivo de bloco como `/dev/sdb1`.
- **Grupo de Volumes (VG – Volume Group):** Agrupamento de um ou mais PVs que constitui uma reserva de armazenamento comum, da qual se podem alocar volumes lógicos.
- **Volume Lógico (LV – Logical Volume):** Corresponde à unidade que é formatada com um sistema de ficheiros e montada no sistema. Pode ser redimensionada de forma dinâmica sem necessidade de voltar a criar partições no disco.

O LVM permite criar volumes lógicos com nomes personalizados, definir tamanhos ajustados e gerir o armazenamento de forma mais eficiente e modular. A criação de sistemas de ficheiros sobre LVs segue o mesmo processo que num dispositivo físico tradicional, podendo usar-se ferramentas como `mkfs.ext3` ou `mkfs.ext4`.

2.3.3. Análise de Sistemas de Ficheiros

A análise de sistemas de ficheiros é um processo técnico que visa a inspeção da estrutura interna de um sistema de ficheiros, muitas vezes aplicada em contextos de recuperação de dados ou de auditoria forense.

Existem ferramentas específicas que permitem examinar ficheiros de imagem (por exemplo, ficheiros `.img`), como as utilizadas neste trabalho:

- **fsstat:** Mostra estatísticas detalhadas sobre o sistema de ficheiros, incluindo o tipo (ex.: FAT12, ext4), o tamanho dos blocos e a estrutura geral do volume.
- **fls:** Lista os ficheiros e diretórios existentes num volume, incluindo entradas apagadas. É útil para identificar o conteúdo lógico de um sistema de ficheiros, mesmo após operações de eliminação.
- **icat:** Permite extrair o conteúdo de um ficheiro a partir do seu número de bloco ou inode, mesmo que tenha sido apagado. Esta operação é realizada com base na análise dos metadados ainda presentes na estrutura do sistema de ficheiros.

3. Implementação

Esta secção descreve a metodologia e as técnicas utilizadas no desenvolvimento das soluções para os problemas propostos no enunciado do trabalho prático. A implementação foi realizada na linguagem C, utilizando o sistema operativo Linux.

3.1. Parte 1: Implementação dos Comandos de Manipulação de Ficheiros

Para a implementação dos comandos de manipulação de ficheiros, foram utilizadas as *system calls* fornecidas pelo sistema operativo Linux. Segue-se a descrição de como cada comando foi implementado:

3.1.1. Mostra Ficheiro

No ficheiro `mostra.c`, implementou-se a funcionalidade de apresentar o conteúdo de um ou mais ficheiros no ecrã. O programa principal recebe os nomes dos ficheiros a exibir como argumentos da linha de comandos.

Inicialmente, verifica-se se foi fornecido pelo menos um nome de ficheiro como argumento ($\text{argc} < 2$). Em caso negativo, escreve-se uma mensagem de erro no descritor de erro padrão (`stderr`) informando a falta do nome do ficheiro, e o programa termina com o código de saída 1.

O programa itera sobre os argumentos da linha de comandos, a partir do segundo argumento (`argv[1]`). Para cada nome de ficheiro, executam-se as seguintes operações:

Abertura do Ficheiro: Utiliza-se a *system call* `open` para abrir o ficheiro especificado (`argv[i]`) em modo de leitura (`O_RDONLY`). O descritor de ficheiro retornado por `open` é armazenado na variável `fd`.

Verificação da Existência do Ficheiro: Verifica-se o sucesso da chamada `open`. Se `fd` for igual a -1, indica-se um erro na abertura do ficheiro (potencialmente devido à sua inexistência). Nesse caso, escreve-se uma mensagem informativa no descritor de erro padrão (`stderr`) com o nome do ficheiro que não foi possível abrir. A execução do ciclo `for` prossegue para o próximo ficheiro através da instrução `continue`.

Leitura e Escrita do Conteúdo: Caso o ficheiro seja aberto com sucesso, inicia-se um ciclo `while` para ler o seu conteúdo e escrevê-lo no ecrã. A *system call* `read` lê até 100 bytes do ficheiro referenciado por `fd` para o buffer. O número de bytes lidos é armazenado na variável `bytes_read`. Enquanto `read` retornar um valor superior a 0 (indicando leitura de bytes), a chamada `write` escreve `bytes_read` do buffer para o descritor de saída padrão (`STDOUT_FILENO`), correspondente ao ecrã. O ciclo continua até que `read` retorne 0 (fim do ficheiro) ou um valor negativo (erro de leitura).

3.1.2. Cópia Ficheiro

O programa `copia.c` implementa a funcionalidade de copiar o conteúdo de um ficheiro para um novo ficheiro, adicionando o sufixo ".copia" ao seu nome original. Para realizar as operações de leitura e escrita de ficheiros, utilizam-se *system calls*.

Funcionalidades:

Validação de Argumentos: Verifica-se se o número de argumentos fornecidos na linha de comandos é o correto. Caso não seja fornecido um nome de ficheiro ou se forem fornecidos argumentos em excesso, apresenta-se uma mensagem de erro no descritor de erro padrão (`stderr`) e a execução do programa é terminada.

Abertura do Ficheiro de Origem: Tenta-se abrir o ficheiro especificado como argumento para leitura (`O_RDONLY`). Se a *system call* `open` falhar, indicando que o ficheiro não existe, escreve-se uma mensagem de erro em `stderr` e o programa é terminado.

Leitura do Conteúdo: O conteúdo do ficheiro de origem é lido para um buffer na memória. A *system call* `read` retorna o número de bytes lidos, o qual é armazenado na variável `bytes_read`.

Criação do Nome do Ficheiro de Destino: O nome do ficheiro de destino é criado através da concatenação do nome do ficheiro original com o sufixo ".copia".

Abertura do Ficheiro de Destino: O ficheiro de destino é criado e aberto para escrita (`O_WRONLY | O_CREAT | O_TRUNC`). As permissões do ficheiro são definidas como `0644` (leitura e escrita para o proprietário, leitura para o grupo e outros). A flag `O_TRUNC` garante que, se o ficheiro já existir, o seu conteúdo será apagado antes da escrita do novo conteúdo.

Escrita do Conteúdo: O conteúdo lido do ficheiro de origem é escrito no ficheiro de destino. A *system call* `write` utiliza o valor de `bytes_read` para assegurar que apenas os dados lidos são escritos.

Fecho dos Ficheiros: Os descritores de ficheiro são fechados utilizando a *system call* `close`.

Utilização:

O programa é executado a partir da linha de comandos com a seguinte sintaxe:

Bash: `./copia <nome_ficheiro_origem>`

Onde `<nome_ficheiro_origem>` é o nome do ficheiro que se pretende copiar.

Exemplos:

- Se o utilizador executar `./copia texto.txt`, o programa criará um novo ficheiro chamado `texto.txt.copia` com o conteúdo de `texto.txt`.
- Se o ficheiro `texto.txt` não existir, o programa apresentará uma mensagem de erro e não criará nenhum ficheiro.

3.1.3. Acrescenta Origem Destino

O programa *acrescenta.c* implementa a funcionalidade de anexar o conteúdo de um ficheiro de origem ao final de um ficheiro de destino.

Funcionalidades:

Validação de Argumentos: Verifica-se a correção do número de argumentos fornecidos na linha de comandos.

Caso o número de argumentos seja inferior a 3 (omissão dos nomes dos ficheiros de origem e/ou destino), escreve-se uma mensagem de erro em *stderr* e o programa termina com o código de erro 1.

Se o número de argumentos exceder 3 (argumentos em excesso), regista-se uma mensagem de erro em *stderr* e o programa finaliza com o código de erro 2.

Cálculo do Comprimento dos Nomes dos Ficheiros: Determina-se o comprimento das strings dos nomes dos ficheiros de origem e destino através de um ciclo *while*, contando os caracteres até ao terminador nulo (`\0`). Este cálculo assegura a correta inclusão dos nomes dos ficheiros nas mensagens de erro.

Abertura do Ficheiro de Origem: Proceda-se à abertura do ficheiro de origem para leitura (`O_RDONLY`).

Se a *system call* `open` falhar (retornando -1), regista-se uma mensagem de erro em *stderr*, incluindo o nome do ficheiro de origem, e o programa é terminado com o código de erro 3.

Abertura do Ficheiro de Destino: Tenta-se abrir o ficheiro de destino para escrita, utilizando o modo `O_WRONLY | O_APPEND`.

A flag `O_WRONLY` permite exclusivamente a escrita no ficheiro.

A flag `O_APPEND` garante que todas as operações de escrita são efetuadas no final do ficheiro, preservando o conteúdo pré-existente.

Se a *system call* `open` falhar (retornando -1), emite-se uma mensagem de erro em *stderr*, contendo o nome do ficheiro de destino, e o programa encerra com o código de erro 4.

Cópia do Conteúdo: O conteúdo do ficheiro de origem é copiado para o ficheiro de destino através de um ciclo *while*.

A *system call* `read` lê até 5 bytes do ficheiro de origem para o buffer.

A *system call* `write` escreve o número de bytes lidos (`bytes_read`) no ficheiro de destino.

O ciclo prossegue até que `read` retorne 0 (fim do ficheiro de origem) ou um valor negativo (erro).

Fecho dos Ficheiros: Os descritores de ficheiro são fechados com a *system call* `close`, assegurando a libertação de todos os recursos alocados.

Retorno: O programa retorna o valor 0, indicando uma execução bem-sucedida.

Utilização:

O programa é executado a partir da linha de comandos com a seguinte sintaxe:

Bash: `./acrescenta <ficheiro_origem> <ficheiro_destino>`

Onde `<ficheiro_origem>` é o nome do ficheiro cujo conteúdo será copiado, e `<ficheiro_destino>` é o nome do ficheiro ao qual o conteúdo será acrescentado.

Exemplos:

- Se o utilizador executar `./acrescenta file1.txt file2.txt`, o conteúdo de `file1.txt` será acrescentado ao final de `file2.txt`.
- Se `file1.txt` ou `file2.txt` não existirem, o programa apresentará uma mensagem de erro e não modificará nenhum ficheiro (no caso de `file2.txt` não existir, ele também não será criado).

3.1.4. Conta Ficheiro

O programa `conta.c` implementa a funcionalidade de contabilizar o número de linhas existentes num ficheiro de texto.

Funcionalidades:

Validação de Argumentos: Verifica-se a validade do número de argumentos fornecidos na linha de comandos.

Caso não seja fornecido um nome de ficheiro, regista-se uma mensagem de erro em `stderr` e o programa termina com o código de erro 1.

Se for fornecido mais de um nome de ficheiro, emite-se uma mensagem de erro em `stderr` e o programa finaliza com o código de erro 1.

Cálculo do Comprimento do Nome do Ficheiro: Determina-se o comprimento da string do nome do ficheiro através de um ciclo `while`, contando os caracteres até ao terminador nulo (`\0`). Este procedimento garante a correta inclusão do nome do ficheiro nas mensagens de erro.

Abertura do Ficheiro: Procede-se à abertura do ficheiro especificado para leitura (`O_RDONLY`).

Se a `system call open` falhar (retornando -1), indicando a inexistência do ficheiro, escreve-se uma mensagem de erro em `stderr`, incluindo o nome do ficheiro, e o programa é terminado com o código de erro 1.

Contagem de Linhas: O conteúdo do ficheiro é lido para um buffer e contabiliza-se o número de ocorrências do caractere de nova linha (`\n`).

A `system call read` é utilizada num ciclo `while` para ler porções do ficheiro para o buffer. Para cada porção lida, itera-se sobre os bytes lidos e incrementa-se um contador (linhas) sempre que se encontra um caractere `\n`.

Conversão do Número de Linhas para String: O número de linhas (inteiro) é convertido para uma string.

Utiliza-se um ciclo `do-while` para obter os dígitos do número, convertê-los para caracteres (adicionando o valor do caractere '0') e armazená-los num array (numero). A string é construída por ordem inversa.

Inversão da String: A string que representa o número de linhas é invertida.

Cria-se um *array* (invertido) e copiam-se os caracteres do *array* (numero) para o *array* invertido na ordem inversa.

Escrita do Resultado: O número de linhas é escrito para a saída padrão (*stdout*). Utiliza-se a *system call* *write* para escrever uma mensagem e a *string* que representa o número de linhas.

Fecho do Ficheiro: O descritor do ficheiro é fechado através da *system call* *close*.

Retorno: O programa retorna o valor 0, indicando uma execução bem-sucedida.

Utilização:

O programa é executado a partir da linha de comandos com a seguinte sintaxe:

Bash: `./conta <nome_ficheiro>`

Onde `<nome_ficheiro>` é o nome do ficheiro cujo número de linhas se pretende contar.

Exemplos:

- Se o utilizador executar `./conta file.txt`, o programa apresentará o número de linhas em `file.txt`.
- Se `file.txt` não existir, o programa apresentará uma mensagem de erro.

3.1.5. Apaga Ficheiro

O programa *apaga.c* implementa a funcionalidade de remover um ficheiro.

Funcionalidades:

Validação de Argumentos: Verifica-se a correção do número de argumentos fornecidos na linha de comandos.

Caso não seja fornecido um nome de ficheiro, regista-se uma mensagem de erro em *stderr* e o programa termina com o código de erro 1.

Se for fornecido mais de um nome de ficheiro, emite-se uma mensagem de erro em *stderr* e o programa finaliza com o código de erro 1.

Cálculo do Comprimento do Nome do Ficheiro: Determina-se o comprimento da *string* do nome do ficheiro através de um ciclo *while*, contando os caracteres até ao terminador nulo (`\0`). Este procedimento assegura a correta apresentação do nome do ficheiro nas mensagens ao utilizador.

Confirmação do Utilizador: Solicita-se confirmação ao utilizador antes de proceder à eliminação do ficheiro.

Apresenta-se uma mensagem na saída padrão (*stdout*) questionando o utilizador sobre a intenção de eliminar o ficheiro.

Lê-se a resposta do utilizador da entrada padrão (*stdin*), aguardando um 'S' ou 's' para confirmação.

O *buffer* de entrada é limpo para garantir que quaisquer caracteres adicionais sejam descartados.

Se a resposta não for 'S' nem 's', escreve-se uma mensagem em *stderr* indicando o cancelamento da eliminação e o programa termina com o código de erro 1.

Verificação da Existência do Ficheiro: Tenta-se abrir o ficheiro para leitura (`O_RDONLY`) com o objetivo de verificar a sua existência.

Se a *system call open* falhar (retornando -1), regista-se uma mensagem de erro em *stderr* informando que o ficheiro não existe e o programa termina com o código de erro 1.

Eliminação do Ficheiro: O ficheiro é removido utilizando a *system call unlink*.

Se a *system call unlink* falhar (retornando -1), regista-se uma mensagem de erro em *stderr* indicando uma falha ao apagar o ficheiro e o programa termina com o código de erro 1.

Confirmação de Sucesso: Se o ficheiro for removido com sucesso, apresenta-se uma mensagem na saída padrão (*stdout*) confirmando a eliminação e mostrando o nome do ficheiro apagado.

Fecho do Ficheiro (Opcional): Embora o ficheiro já tenha sido removido, o descritor do ficheiro é fechado através da *system call close*. Apesar de não ser estritamente necessário após o *unlink*, considera-se uma boa prática para libertar recursos.

Retorno: O programa termina com o código de saída 0, indicando sucesso.

Utilização:

O programa é executado a partir da linha de comandos com a seguinte sintaxe:

Bash: `./apaga <nome_ficheiro>`

Onde `<nome_ficheiro>` é o nome do ficheiro que se pretende apagar.

Exemplos:

- Se o utilizador executar `./apaga important.txt`, o programa solicitará confirmação e, se confirmado, apagará o ficheiro `important.txt`.
- Se o ficheiro `important.txt` não existir, o programa apresentará uma mensagem de erro e não tentará apagar nada.

3.1.6. Informa Ficheiro

O programa *informa.c* tem como objetivo apresentar informação detalhada sobre um ficheiro. Para tal, recorre-se exclusivamente a *system calls*.

Funcionalidades:

O programa apresenta as seguintes informações sobre o ficheiro:

- **Tipo de ficheiro:** Identifica-se se o ficheiro é um ficheiro regular, diretório, *link* simbólico, ficheiro especial de caractere, ficheiro especial de bloco, FIFO (*pipe*) ou *socket*.
- **Número do i-node:** Apresenta-se o número do i-node, que constitui um identificador único para o ficheiro no sistema de ficheiros.
- **Identificador do utilizador proprietário (UID):** Mostra-se o UID do utilizador que detém a propriedade do ficheiro.

- **Datas:** Apresentam-se as datas de criação, último acesso e última modificação do ficheiro. As datas são convertidas de *timestamp* para o formato "dd mês aaaa hh:mm:ss".

Implementação:

O programa utiliza a *system call lstat* para obter informações sobre o ficheiro. A função *lstat* é semelhante a *stat*, mas, no caso de um *link* simbólico, retorna informação sobre o próprio *link*, e não sobre o ficheiro para o qual o *link* aponta.

A função *escreve_data* é responsável por converter um *timestamp* (*timestamp* é um número inteiro que representa o número de segundos desde um ponto de referência no tempo, neste caso a Época Unix, 1 de janeiro de 1970 às 00:00:00 UTC) para o formato de data e hora pretendido. Esta função efetua cálculos para determinar o ano, mês, dia, hora, minuto e segundo correspondentes ao *timestamp*.

As funções *myitoa*, *mywrite* e *myerror* (definidas no ficheiro *funcoes.c*) são utilizadas para converter inteiros para *strings*, escrever para a saída padrão e escrever para a saída de erro, respetivamente. Estas funções também são implementadas utilizando *system calls*.

Utilização:

O programa é invocado a partir da linha de comandos com a seguinte sintaxe:

./informa <nome_ficheiro>

Onde *<nome_ficheiro>* é o nome do ficheiro sobre o qual se pretende obter informações. Se o número de argumentos fornecidos ao programa for incorreto, é apresentada uma mensagem de erro no descritor de erro padrão (*stderr*).

Exemplo:

Para obter informações sobre um ficheiro chamado "exemplo.txt", o comando seria:

Bash: ./informa exemplo.txt

Este comando apresentará na saída padrão (*stdout*) as informações sobre o tipo, i-node, UID e datas do ficheiro "exemplo.txt". Se ocorrer um erro ao aceder ao ficheiro (por exemplo, se o ficheiro não existir), será apresentada uma mensagem de erro em *stderr*.

3.1.7. Lista Diretoria

O programa *lista.c* implementa a funcionalidade de listar o conteúdo de um diretório, apresentando ficheiros e subdiretórios e distinguindo-os textualmente.

Funcionalidades:

Validação de Argumentos: Verifica-se o número de argumentos fornecidos na linha de comandos.

Caso sejam fornecidos mais de dois argumentos, invoca-se a função *myerror* para escrever uma mensagem de erro em *stderr*.

Determinação do Caminho do Diretório: Determina-se o caminho do diretório a ser listado.

Se nenhum caminho for fornecido (apenas o nome do programa), utiliza-se o diretório atual (".") como padrão.

Se for fornecido um caminho como argumento, aplica-se o caminho especificado.

Abertura do Diretório: Abre-se o diretório especificado utilizando a função *opendir*. Se a abertura do diretório falhar (retornando NULL), invoca-se a função *myerror* para escrever uma mensagem de erro em *stderr* e o programa termina.

Leitura das Entradas do Diretório: As entradas do diretório são lidas através da função *readdir* num ciclo *while*.

A função *readdir* retorna um apontador para uma estrutura *dirent* que contém informações sobre cada entrada (ficheiro ou subdiretório) no diretório.

Ignorar "." e "..": As entradas "." (diretório atual) e ".." (diretório pai) são ignoradas para evitar ciclos infinitos e entradas irrelevantes.

Construção do Caminho Absoluto: Para cada entrada, constrói-se o caminho absoluto completo para a entrada.

O caminho do diretório base é copiado para a variável *caminhoabsoluto*.

Adiciona-se uma barra ("/") ao final do caminho, se necessário.

O nome da entrada (ficheiro ou subdiretório) é concatenado ao caminho absoluto.

Verificação do Tipo de Entrada: Verifica-se se a entrada é um diretório ou um ficheiro através da função *is_dir*.

A função *is_dir* utiliza a *system call stat* para obter informações sobre o ficheiro e verifica o campo *st_mode* para determinar se é um diretório (S_ISDIR).

Escrita da Informação: O tipo da entrada e o nome são escritos para a saída padrão (*stdout*) utilizando a função *mywrite*.

Se for um diretório, escreve-se "[DIR] nome_da_entrada".

Se for um ficheiro, escreve-se "[FILE] nome_da_entrada".

Fecho do Diretório: O diretório é fechado utilizando a função *closedir*.

Retorno: O programa retorna 0 para indicar execução bem-sucedida.

Funções Auxiliares (*funcoes.c*):

*mywrite(const char *s)*: Escreve uma *string* para a saída padrão (*stdout*) utilizando a *system call write*.

*myerror(char *s)*: Escreve uma *string* para a saída de erro padrão (*stderr*) utilizando a *system call write*.

*is_dir(char *c)*: Verifica se um caminho representa um diretório utilizando a *system call stat* e a macro S_ISDIR.

Utilização:

O programa é executado a partir da linha de comandos com a seguinte sintaxe:

Bash: ./lista [diretorio]

Onde [diretorio] é o caminho do diretório a ser listado (opcional). Se omitido, o diretório atual será listado.

Exemplos:

- Se o utilizador executar `./lista`, o programa listará o conteúdo do diretório atual.
- Se o utilizador executar `./lista /home/utilizador/Documentos`, o programa listará o conteúdo do diretório `/home/utilizador/Documentos`.

3.2. Parte 2 - Implementação de um interpretador de linha de comandos

Com o objetivo de explorar diferentes abordagens à implementação de um interpretador de comandos e aprofundar a compreensão das *system calls*, optou-se por desenvolver duas implementações distintas.

Apresenta-se, assim, duas soluções: uma que recorre a funções da *LibC* conforme o que foi solicitado no enunciado, e outra que utiliza exclusivamente *system calls*, procurando um controlo mais direto sobre o sistema operativo.

3.2.1. Interpretador com Funções da *LibC*

Esta versão do interpretador foi desenvolvida com recurso às funcionalidades disponibilizadas pela biblioteca padrão da linguagem C (*libc*), conforme definido no enunciado. Esta versão recorre a funções como *fgets()* para leitura do input, *strtok()* para a divisão de argumentos, *strcasecmp()* para comparação de *strings*, e *printf()* para a escrita no ecrã.

O funcionamento do interpretador assenta num ciclo infinito, interrompido apenas pela introdução do comando 'termina'. A cada iteração, é apresentado ao utilizador o símbolo %, sinalizando que o sistema se encontra pronto para receber um novo comando.

Após a leitura da linha de comandos introduzida, esta é tratada de forma a remover o caractere de nova linha ('\n'), seguido da verificação se corresponde à palavra-chave de terminação. Caso não seja esse o caso, o input é segmentado nos respetivos argumentos utilizando *strtok()*, sendo os mesmos organizados num *array* de *strings* que é posteriormente passado à função *execvp()*.

A execução do comando ocorre num processo filho criado através da chamada *fork()*. O processo pai suspende a sua execução até que o filho termine, utilizando a função *wait()*. No final de cada execução, é apresentada uma mensagem ao utilizador a indicar a conclusão do comando.

Este comportamento replica, de forma simplificada, o funcionamento de um interpretador de comandos típico, demonstrando o uso combinado de chamadas de criação de processos (*fork()*), execução (*execvp()*), e sincronização (*wait()*), juntamente com funções utilitárias de alto nível para tratamento de *strings* e input/output.

3.2.2. Interpretador com *System Calls*

Como forma de aprofundar o domínio das funcionalidades oferecidas pelo sistema operativo, foi desenvolvida uma segunda versão do interpretador de comandos, desta vez sem o uso da biblioteca padrão da linguagem C. Esta versão foi concebida com o intuito de utilizar exclusivamente *system calls* e funções auxiliares desenvolvidas para substituição das funcionalidades da *libc*, nomeadamente para escrita (*mywrite()*) e cálculo de comprimento de *strings* (*my_len()*).

Esta abordagem exigiu um controlo mais rigoroso da leitura e manipulação de dados, obrigando à implementação de funcionalidades básicas de forma manual. A leitura da

linha de comandos é feita através da chamada *read()* diretamente a partir do descritor de ficheiro padrão (*stdin*), e a escrita é realizada com *mywrite()*, que internamente invoca *write()* do sistema operativo. Não são utilizadas funções como *printf()* nem *fgets()*.

À semelhança da versão anterior, o interpretador apresenta ao utilizador o símbolo % e aguarda a introdução de um comando. A linha introduzida é lida para um buffer e o caractere de nova linha é substituído por um terminador de *string* ('\0'). A deteção da palavra-chave 'termina' é realizada de forma manual, através da comparação direta de caracteres, sem uso de funções de comparação de *strings*.

A criação de um novo processo continua a ser realizada com *fork()*, sendo que o processo filho executa o comando introduzido usando *execvp()*. Caso a execução falhe, é apresentada uma mensagem de erro utilizando *mywrite()*.

Os argumentos do comando são extraídos separando o conteúdo do buffer original sempre que ocorre um espaço em branco. Este processo é realizado manualmente, convertendo o buffer numa sequência de apontadores para *strings* terminadas em '\0'.

3.3. Parte 3A - Gestão de Sistemas de Ficheiros

Esta secção do trabalho prático demonstra a criação e configuração de sistemas de ficheiros em Linux, seguindo os passos de adicionar um disco virtual, criar partições e volumes lógicos (LVM), e formatá-los com diferentes tipos de sistemas de ficheiros.

3.3.1. Adicionar um Disco Virtual e Criar uma Partição

Foi adicionado um novo disco virtual com 10GB de espaço alocado dinamicamente ao servidor virtual (Figura 1 e Figura 2).

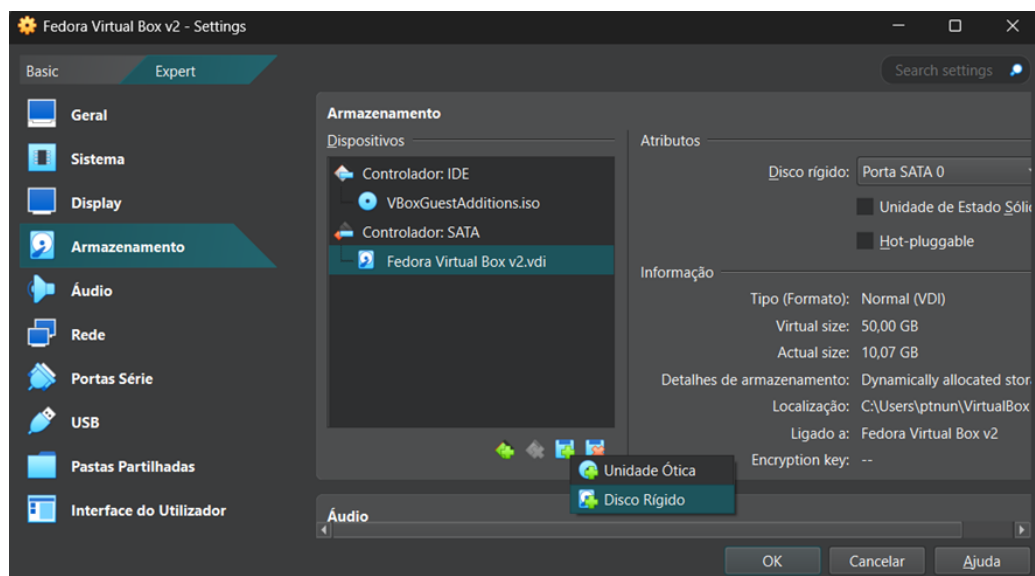


Figura 1

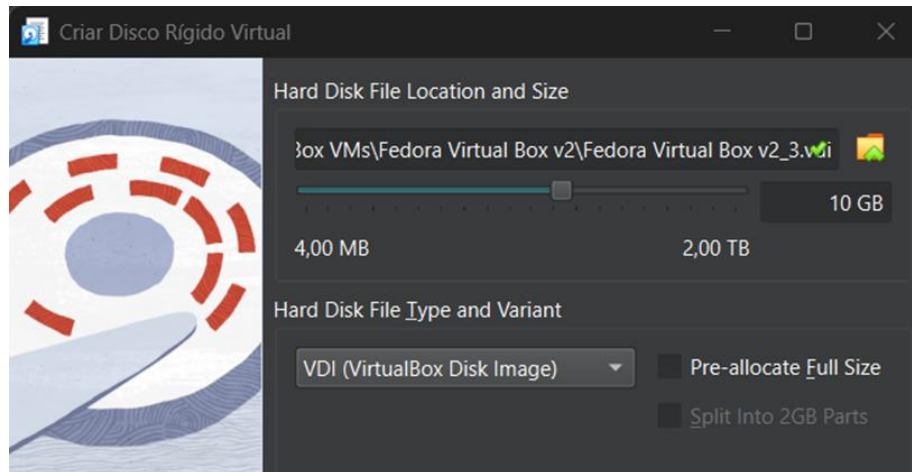


Figura 2

O disco foi identificado usando o comando `sudo fdisk -l /dev/sdb` (Figura 3).

```
ptnunooliveira@vbox:~$ sudo fdisk -l /dev/sdb
Disk /dev/sdb: 10 GiB, 10737418240 bytes, 20971520 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

Figura 3

Utilizou-se o comando `sudo fdisk /dev/sdb` para criar uma nova partição.

- Dentro do `fdisk`, os passos foram:
 - `n` para nova partição (Figura 4).

```
ptnunooliveira@vbox:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.40.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x81110701.

Command (m for help): n
```

Figura 4

- o p para partição primária (Figura 5).

```
ptnunooliveira@vbox:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.40.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x81110701.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
```

Figura 5

- o 1 para o número da partição (Figura 6).

```
ptnunooliveira@vbox:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.40.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x81110701.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
```

Figura 6

- o A partição criada (/dev/sdb1) (Figura 7 e Figura 8).

```
ptnunooliveira@vbox:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.40.4).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x81110701.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-20971519, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-20971519, default 20971519):

Created a new partition 1 of type 'Linux' and of size 10 GiB.
```

Figura 7

```

Disk /dev/sdb: 10 GiB, 10737418240 bytes, 20971520 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x81110701

Device      Boot Start      End  Sectors  Size Id Type
/dev/sdb1                2048 20971519 20969472   10G 83 Linux

```

Figura 8

3.3.2. Criação de Volumes Lógicos (LVM)

- Foi criado um *Physical Volume* (PV) usando o comando `sudo pvcreate /dev/sdb1` (Figura 9).

```

ptnunooliveira@vbox:~$ sudo pvcreate /dev/sdb1
[sudo] senha para ptnunooliveira:
Physical volume "/dev/sdb1" successfully created.

```

Figura 9

- Verificou-se o PV criado com o comando `sudo pvs` (Figura 10).

```

ptnunooliveira@vbox:~$ sudo pvs
PV          VG Fmt  Attr PSize  PFree
/dev/sdb1   lvm2 ---  <10,00g <10,00g

```

Figura 10

- Foi criado um *Volume Group* (VG) chamado "vgs_proj" usando o comando `sudo vgcreate vgs_proj /dev/sdb1` (Figura 11).

```

ptnunooliveira@vbox:~$ sudo vgcreate vgs_proj /dev/sdb1
Volume group "vgs_proj" successfully created

```

Figura 11

- Verificou-se o VG com o comando `sudo pvs` (Figura 12).

```

ptnunooliveira@vbox:~$ sudo pvs
PV          VG      Fmt  Attr PSize  PFree
/dev/sdb1   vgs_proj lvm2 a--  <10,00g <10,00g

```

Figura 12

- Foram criados dois *Logical Volumes* (LV):
 - `sudo lvcreate -L 5G -n lv_proj1 vgs_proj` (Figura 12) para criar o primeiro LV com 5GB (Figura 13).
 - Inicialmente, tentou-se criar o segundo LV também com 5GB, mas não havia espaço suficiente (Figura 13).

```
ptnunooliveira@vbox:~$ sudo lvcreate -L 5G -n lv_proj1 vgs_proj
Logical volume "lv_proj1" created.
ptnunooliveira@vbox:~$ sudo lvcreate -L 5G -n lv_proj2 vgs_proj
Volume group "vgs_proj" has insufficient free space (1279 extents): 1280 required.
```

Figura 13

Nota adicional: Embora o *Volume Group* ('vgs_proj') tenha sido criado com 10GB de capacidade, a tentativa de criar dois LVs de 5GB cada falhou devido à gestão interna do espaço pelo LV (Figura 13). O LV reserva espaço para *metadados* e aloca espaço em unidades ('*extents*'), o que causa arredondamentos e reduz o espaço efetivamente disponível para os LVs.

Para resolver este problema, foi decidido reduzir ligeiramente o tamanho do segundo LV ('lv_proj2') para 4.99GB. Esta pequena redução permitiu que o LV alocasse o espaço necessário dentro das suas restrições, resultando na criação bem-sucedida do segundo LV (Figura 14)."

```
ptnunooliveira@vbox:~$ sudo lvcreate -L 4.99G -n lv_proj2 vgs_proj
Rounding up size to full physical extent 4,99 GiB
Logical volume "lv_proj2" created.
```

Figura 14

3.3.3. Criação de Sistemas de Ficheiros

- Foi criado um sistema de ficheiros ext4 no LV "lv_proj1" usando o comando `sudo mkfs.ext4 /dev/vgs_proj/lv_proj1` (Figura 15).

```
ptnunooliveira@vbox:~$ sudo mkfs.ext4 /dev/vgs_proj/lv_proj1
mke2fs 1.47.1 (20-May-2024)
A criar sistema de ficheiros com 1310720 4k blocos e 327680 inodes
UUID do sistema de ficheiros: d2344d3b-5f49-4bc2-8670-cefd26d76334
Seguranças de super-blocos armazenadas em blocos:
    32768, 98304, 163840, 229376, 294912, 819200, 884736

A alocar tabelas de grupo: feito
A escrever tabelas de inodes: feito
A criar diário (16384 blocos): feito
A escrever super-blocos e informação de contabilidade do sistema de ficheiros:
feito
```

Figura 15

- Foi criado um sistema de ficheiros ext3 no LV "lv_proj2" usando o comando `sudo mkfs.ext3 /dev/vgs_proj/lv_proj2` (Figura 16).

```
ptnunooliveira@vbox:~$ sudo mkfs.ext3 /dev/vgs_proj/lv_proj2
mke2fs 1.47.1 (20-May-2024)
A criar sistema de ficheiros com 1308672 4k blocos e 327680 inodes
UUID do sistema de ficheiros: df40d59d-120f-4a3d-8a0a-e7f228b49a49
Seguranças de super-blocos armazenadas em blocos:
    32768, 98304, 163840, 229376, 294912, 819200, 884736

A alocar tabelas de grupo: feito
A escrever tabelas de inodes: feito
A criar diário (16384 blocos): feito
A escrever super-blocos e informação de contabilidade do sistema de ficheiros:
feito
```

Figura 16

3.3.4. Montagem dos Sistemas de Ficheiros

- Foram criados os diretórios de montagem:
 - `sudo mkdir /mnt/lv_proj1` (Figura 17) para o LV ext4.

```
ptnunooliveira@vbox:~$ sudo mkdir /mnt/lv_proj1
```

Figura 17

- `sudo mkdir /mnt/lv_proj2` (Figura 18) para o LV ext3.

```
ptnunooliveira@vbox:~$ sudo mkdir /mnt/lv_proj2
```

Figura 18

- Os sistemas de ficheiros foram montados:
 - `sudo mount /dev/vgs_proj/lv_proj1 /mnt/lv_proj1` (Figura 19).

```
ptnunooliveira@vbox:~$ sudo mount /dev/vgs_proj/lv_proj1 /mnt/lv_proj1
```

Figura 19

- `sudo mount /dev/vgs_proj/lv_proj2 /mnt/lv_proj2` (Figura 20).

```
ptnunooliveira@vbox:~$ sudo mount /dev/vgs_proj/lv_proj2 /mnt/lv_proj2
```

Figura 20

- A montagem foi verificada usando o comando `sudo mount` (Figura 21 e Figura 22).

```
/dev/mapper/vgs_proj-lv_proj1 on /mnt/lv_proj1 type ext4 (rw,relatime,seclabel)
```

Figura 21

```
/dev/mapper/vgs_proj-lv_proj2 on /mnt/lv_proj2 type ext3 (rw,relatime,seclabel)
```

Figura 22

- Verificação do espaço dos *logical volumes*, com o comando `df -h` (Figura 23).

```
/dev/mapper/vgs_proj-lv_proj1 4,9G 1,3M 4,6G 1% /mnt/lv_proj1
/dev/mapper/vgs_proj-lv_proj2 4,9G 92K 4,6G 1% /mnt/lv_proj2
```

Figura 23

Criação de Ficheiro e Ajuste de Permissões

Foi criado um ficheiro na diretoria `/mnt/lv_proj1` (onde o sistema ext4 foi montado) com o nome "31550-31029-31103.txt" (Figura 24). Foi utilizado o comando `sudo touch` pois sem ele não existiam permissões para a operação.

```
ptnunooliveira@vbox:/mnt/lv_proj1$ sudo touch 31550-31029-31103.txt
```

Figura 24

Verificação do ficheiro criado (Figura 25).

```
ptnunooliveira@vbox:/mnt/lv_proj1$ ls
31550-31029-31103.txt
```

Figura 25

Permissões atuais (Figura 26).

```
-rw-r--r--. 1 root root 0 mai 6 14:35 31550-31029-31103.txt
```

Figura 26

As permissões do ficheiro foram ajustadas para leitura e escrita para o dono, nenhuma para o grupo e leitura para outros (Figura 27 e Figura 28).

```
ptnunooliveira@vbox:/mnt/lv_proj1$ sudo chmod 604 /mnt/lv_proj1/31550-31029-31103.txt
```

Figura 27

```
ptnunooliveira@vbox:/mnt/lv_proj1$ ls -la
total 0
drwxr-xr-x. 1 root root 42 mai 6 14:35 .
drwxr-xr-x. 1 root root 42 mai 6 13:31 ..
-rw----r--. 1 root root 0 mai 6 14:35 31550-31029-31103.txt
```

Figura 28

3.4. Parte 3B - Análise ao Sistemas de Ficheiros

Esta secção do trabalho prático foca-se na análise de um sistema de ficheiros contido num ficheiro de imagem (*fs.img*).

3.4.1. Análise Geral do Sistema de Ficheiros

O comando *fsstat -o 0 fs.img* é utilizado para obter detalhes gerais sobre o sistema de ficheiros. Este comando fornece informações como o tipo de sistema de ficheiros, o tamanho do setor e a disposição dos dados. A Figura 29 mostra o resultado do comando *fsstat*.

```
user@DELL7490:/mnt/c/S0$ fsstat -o 0 fs.img
FILE SYSTEM INFORMATION
-----
File System Type: FAT12

OEM Name: mkfs.fat
Volume ID: 0xaa724ea
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory):
File System Type Label: FAT12

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2047
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 2
* FAT 1: 3 - 4
* Data Area: 5 - 2047
** Root Directory: 5 - 36
** Cluster Area: 37 - 2044
** Non-clustered: 2045 - 2047

METADATA INFORMATION
-----
Range: 2 - 32694
Root Directory: 2

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 2048
Total Cluster Range: 2 - 503

FAT CONTENTS (in sectors)
-----
65-68 (4) -> EOF
```

Figura 29

A análise revela que o sistema de ficheiros é do tipo FAT12 e que o tamanho do setor é de 512 bytes.

3.4.2. Listagem de Ficheiros e Pastas

O comando `fls -o 0 fs.img` é utilizado para listar os ficheiros e pastas presentes na imagem do sistema de ficheiros. O parâmetro `-o 0` especifica que a leitura deve começar no início da imagem (offset zero).

A ferramenta `fls` identifica tanto os ficheiros ativos como os ficheiros que foram apagados, marcando os ficheiros apagados com um asterisco antes do número do bloco.

A Figura 30 apresenta a saída do comando `fls`.

```
user@DELL7490:/mnt/c/SO$ fls -o 0 fs.img
r/r * 4:      ipca.txt
r/r * 5:      _ICHEI~1.SWP
r/r 7:  ficheiro1
r/r * 9:      ficheiro2
r/r * 10:     _ICHEI~2.SWP
v/v 32691:    $MBR
v/v 32692:    $FAT1
v/v 32693:    $FAT2
V/V 32694:    $OrphanFiles
user@DELL7490:/mnt/c/SO$ |
```

Figura 30

A listagem indica que o único ficheiro ativo se encontra no bloco 7. No entanto, se considerarmos os ficheiros apagados, o ficheiro "ipca.txt" estava localizado no bloco 4.

3.4.3. Recuperação do Conteúdo de um Ficheiro

Para extrair o conteúdo do ficheiro "ipca.txt", utiliza-se o comando `icat -o 0 fs.img 4`. O número 4 representa o número do bloco (ou *inode*) onde o ficheiro estava localizado. A Figura 31 mostra o conteúdo do ficheiro "ipca.txt" enviado para o stdout. Para recuperar o ficheiro podíamos utilizar o comando: "icat -o 0 fs.img 4 > ficheiro_extraido.txt" mas é arriscado porque poderíamos estar a escrever por cima deste ou outro ficheiro apagado. Caso fosse necessário recuperar o ficheiro deveríamos fazê-lo para uma unidade distinta.

```
user@DELL7490:/mnt/c/SO$ icat -o 0 fs.img 4
Este é o conteúdo do ficheiro ipca.txt
user@DELL7490:/mnt/c/SO$
```

Figura 31

O conteúdo do ficheiro é: "Este é o conteúdo do ficheiro ipca.txt".

4. Conclusão

O desenvolvimento do presente trabalho prático permitiu consolidar os conhecimentos teóricos adquiridos na unidade curricular, através da sua aplicação em contextos práticos e realistas. A implementação de comandos de manipulação de ficheiros com recurso exclusivo a *system calls* proporcionou uma compreensão mais aprofundada sobre o funcionamento interno do sistema operativo, em particular no que respeita à gestão de ficheiros e à utilização de descritores de ficheiros.

A criação de um interpretador de linha de comandos, em duas versões distintas (com funções da *LibC* e exclusivamente com *system calls*), evidenciou a importância do controlo direto sobre o processo de execução de comandos, bem como a necessidade de uma gestão rigorosa de apontadores, buffers e fluxos de entrada/saída. A abordagem baseada em *system calls* revelou-se particularmente desafiante, exigindo a construção de funcionalidades de base para operações comuns, o que contribuiu significativamente para o desenvolvimento de competências de baixo nível em C.

No âmbito da gestão de sistemas de ficheiros, foram realizados procedimentos completos de criação e configuração de partições, volumes lógicos (LVM) e sistemas de ficheiros, permitindo explorar os comandos e ferramentas típicas de administração de sistemas Linux. A análise de uma imagem de sistema de ficheiros, demonstrou a utilidade das estruturas internas dos sistemas de ficheiros, como blocos, *inodes* e diretórios, bem como a possibilidade de recuperar ficheiros eliminados.

De forma global, o trabalho proporcionou uma experiência prática abrangente, centrada nos mecanismos fundamentais que suportam o funcionamento de um sistema operativo moderno.

5. Bibliografia

<https://linux.die.net/man/>

Anexo A-Enunciado do Trabalho Prático



Lic. em Eng. de Sistemas Informáticos
Sistemas Operativos
2024/2025

Enunciado do Trabalho Prático

Este trabalho prático tem como objetivo a aplicação prática dos conceitos de gestão de processos e de ficheiros.

O trabalho deverá ser realizado em grupos de 3 ou 4 elementos, excepcionalmente com outro número de elementos, devidamente justificado e aceite pelo docente.

Parte 1) Implementação um conjunto de comandos para manipulação de ficheiros (10,5 vals)

Implemente os seguintes comandos através de funções de chamada ao sistema (*system calls*) (ver <https://linux.die.net/man/>). Estes comandos devem ser implementados em C (para Linux) e serão invocados através de um interpretador de comandos. Qualquer mensagem de erro deverá ser apresentada no descritor *stderr*, não utilize programação *shell*, funções LibC (*fopen*, *fread*, etc...) ou comandos pré-existentes para executar a funcionalidade pretendida.

- a) **mostra** ficheiro – Este comando deve apresentar no ecrã **todo** o conteúdo do ficheiro indicado como parâmetro. Caso o ficheiro não exista, o comando deve avisar o utilizador que o ficheiro não existe;
- b) **copia** ficheiro – Este comando deve criar um novo ficheiro, cujo nome é “*ficheiro.copia*”, cujo conteúdo é uma cópia de (todo) o conteúdo do ficheiro passado como parâmetro no comando, com o nome *ficheiro*. Caso o ficheiro não exista, deve ser apresentado um aviso ao utilizador;
- c) **acrescenta** origem destino – Este comando deve acrescentar (todo) o conteúdo da “origem” no final do “destino”. Caso algum dos ficheiros não exista, deve ser apresentado um aviso ao utilizador;
- d) **conta** ficheiro – Este comando deve contar o número de linhas existentes num ficheiro. Se o ficheiro não existir, deverá ser indicado ao utilizador uma mensagem de erro;
- e) **apaga** ficheiro – Este comando deve apagar o ficheiro com o nome indicado. No caso de o ficheiro indicado **não** existir, e apenas, deve ser apresentado um aviso ao utilizador;
- f) **informa** ficheiro – Este comando apresenta **apenas** a informação do sistema de ficheiros em relação ao ficheiro indicado, tipo de ficheiro (normal, diretoria, link, etc.), i-node, utilizador dono em formato textual e datas de criação, leitura e modificação em formato textual;

- g) **lista** [directoria] – Este comando deve apresentar uma lista de todas as pastas e ficheiros existentes na directoria indicada ou na directoria atual se não especificada. Adicionalmente, deve distinguir ficheiros simples de directorias através de uma indicação textual.

Parte 2) Implementação de um interpretador de linha de comandos (4 vals)

No sentido de substituir o interpretador de comandos habitual, Bash shell, por um novo interpretador personalizado, deve ser implementada uma aplicação, de nome *interpretador*, cuja função será a leitura de uma sequência de caracteres da consola, e em seguida executar essa sequência como um comando e respetivos argumentos no sistema. O programa deve mostrar o símbolo “%” como indicação de que está pronto para ler um novo comando do utilizador.

O programa deve executar o comando através de primitivas de execução genérica de processos tendo como referência a funcionalidade da função *system(3)*, mas **sem fazer uso da mesma**. Cada comando escrito deve dar origem a um novo processo. Adicionalmente, pode considerar que a execução do interpretador deve ser suspensa até o comando indicado estar concluído. O interpretador deve indicar sempre se o comando concluiu com ou sem sucesso, através do seu código de erro/terminação. O programa deve permitir executar vários comandos sequencialmente, isto é, um a seguir ao outro, até o utilizador indicar o comando especial “termina” que termina esta aplicação.

```
$ ./interpretador
% lista /home/user/Desktop
...
Terminou comando lista com código 0
% apaga /home/user/Desktop/file
...
Terminou comando apaga com código 1
% termina
$
```

Parte 3A) Gestão de Sistemas de Ficheiros (3,5 vals: 0,5+1+1+0,5+0,5)

- a) Num servidor virtual, adicione um disco novo com o tamanho de 10GB (espaço alocado dinamicamente) e crie uma partição.
- b) No disco virtual criado na alínea a), deve criar um volume, que ocupe o espaço todo, e dentro desse volume, deve adicionar dois volumes lógicos, cada um com o tamanho de 5GB.
- c) Nos volumes lógicos criados no passo b), crie um sistema de ficheiros ext4 em um deles e ext3 no outro.
- d) Monte cada um dos sistemas de ficheiros criados em c) nas directorias `/mnt/ext4` e `/mnt/ext3`, respectivamente, ficando persistente a reboots.
- e) Dentro da directoria `/mnt/ext4`, crie um ficheiro com o nome composto pelo grupo dos números de alunos que constituem o trabalho, e a extensão `.txt` (exemplo: `22222-22233-23333-24003.txt`). Esse ficheiro deverá ter, apenas, permissões de escrita e leitura para o dono (que será o utilizador que está a usar o sistema sem ser `root`), o grupo não deve ter qualquer permissão neste ficheiro, e todos os outros devem ter permissão de leitura.

Parte 3B) Análise ao Sistemas de Ficheiros (2 vals: 1+1)

Com base no ficheiro *fs.img* (disponível no moodle), responda às seguintes questões:

- a) Identificar o bloco de início dos ficheiros regulares presentes do sistema de ficheiros
- b) Indique o conteúdo do ficheiro com o nome `ipca.txt`

Em todos os passos deve descrever os passos necessários, completando a sua resposta com os *screenshots* necessários.

Listagem de Chamadas ao Sistema

As chamadas ao sistema que poderão ser utilizadas neste trabalho encontram-se na secção 2 das páginas do Manual de Unix¹.

Gestão de Ficheiros

- open
- read
- write
- close
- dup e dup2
- pipe
- stat(2)
- opendir(3)
- readdir(3)
- closedir(3)

Gestão de Processos

- fork
- execve(2)
- exec(3)
- wait

As seguintes funções **não** devem ser usadas na resolução deste trabalho:

- system, fopen, fclose, fread, fwrite, fseek

¹ <https://linux.die.net/man/2/>